

markpact

Live Contract Generation

PROMPT: custom

Build a todo API

Model: ollama/qwen2.5-coder:14b

0. Meta-Pipeline

Jak powstał ten dokument? human -> marksync -> LLM -> PDF

README.MD
Meta-Pipeline: Jak powstał ten dokument? Zanim powstał ten kontrakt, człowiek uruchomił marksync: ```yaml markpack:orchestration # Meta-pipeline generowania kontraktu pipeline: name: marksync-report-generation steps: - name: user_request actor: human config: action: run_command command: marksync report - name: parse_intent actor: script config: tool: marksync.intent.IntentParser - name: generate_contract actor: llm config: model: ollama/qwen2.5-coder:14b prompt: user_request - name: enrich_orchestration actor: script config: tool: marksync.intent.YAMLGenerator - name: render_pdf actor: script config: tool: marksync.report.ReportPDF - name: generate_report actor: script config: tool: marksync.report.ReportPDF
Kazdy dokument marksync jest wynikiem współpracy: 1. HUMAN - definiuje zamysł (prompt) 2. SCRIPT - orkiestruje proces (marksync) 3. LLM - generuje kontrakt 4. SCRIPT - wzbogaca + renderuje PDF

WALIDACJA + OCZEKIWANIA
[Meta-Pipeline] Aktorzy: human - uruchamia marksync script - marksync CLI llm - generuje kontrakt script - renderuje PDF Przepływ: human -> marksync report marksync -> IntentParser marksync -> LLM (prompt) LLM -> README.md marksync -> YAMLGenerator marksync -> ReportPDF ReportPDF -> PDF output Ten dokument to meta-refleksja: pokazuje proces własnego powstania.
WALIDACJA
[V] human uruchomił marksync marksync report [V] script parsuje intent IntentParser [V] LLM generuje kontrakt ollama/qwen2.5-coder:14b [V] script wzbogaca YAML YAMLGenerator [V] script renderuje PDF ReportPDF

1. Generowanie kontraktu

LLM: ollama/qwen2.5-coder:14b

README.MD

```
# Todo API

A simple FastAPI-based Todo application that allows users to create, retrieve, update

## Features
- Create a new todo item
- Retrieve all todo items
- Update an existing todo item
- Delete a todo item

## API Endpoints
- POST /todos - Create a new todo item
- GET /todos - Retrieve all todo items
- PUT /todos/{id} - Update an existing todo item
- DELETE /todos/{id} - Delete a todo item

---

```yaml markpackt:orchestration
pipeline:
 name: todo-api
 steps:
 - name: validate_input
 actor: script
 - name: ai_review
 actor: llm
 - name: human_approval
 actor: human
 config: {channel: web, timeout: 7200}
 - name: deploy
 actor: script
```

```json markpackt:state
{"phase":"init","health":null,"success_count":0,"error_count":0}
```

```text markpackt:deps python
fastapi
uvicorn
sqlalchemy
pydantic
```

```python markpackt:file path=app/main.py
from fastapi import FastAPI, HTTPException, Path
from pydantic import BaseModel
from typing import List, Optional

app = FastAPI()
```

WALIDACJA + OCZEKIWANIA

Prompt: custom

Build a todo API...

Model: ollama/qwen2.5-coder:14b

Czas: 132845ms

Rozmiar: 2821 znakow

Linie: 112

WALIDACJA

[V]

LLM odpowiedzial

132845ms

[V]

README.md zapisany

generated/live/README.md

# 1b. marksync Orchestration

IntentParser + YAMLGenerator

```
README.MD

Todo API

A simple FastAPI-based Todo application that allows users to create, retrieve, update

Features
- Create a new todo item
- Retrieve all todo items
- Update an existing todo item
- Delete a todo item

API Endpoints
- POST /todos - Create a new todo item
- GET /todos - Retrieve all todo items
- PUT /todos/{id} - Update an existing todo item
- DELETE /todos/{id} - Delete a todo item

```yaml markpact:pipeline
description: Build a todo API
name: build-a-todo-api
steps:
- actor: script
  config:
    check: schema
    name: validate
- actor: script
  config:
    action: deploy
    target: docker
    name: deploy
version: 1.0.0
```

```text markpact:log
[2026-02-19T08:07:54Z] CONTRACT_CREATED: name=build-a-todo-api
```

```yaml markpact:orchestration
pipeline:
  name: todo-api
  steps:
    - name: validate_input
      actor: script
    - name: ai_review
      actor: llm
    - name: human_approval
      actor: human
      config: {channel: web, timeout: 7200}
```

WALIDACJA + OCZEKIWANIA

marksync wzbogaca kontrakt o:
markpact:orchestration -- pipeline krokow
markpact:pipeline -- definicja steps
markpact:state -- stan procesu
markpact:log -- historia zdarzen

service_type: generic
actors: script

WALIDACJA

[V]

 markpact:pipeline injected

[V]

 markpact:log injected

2. Parsowanie blokow

parse_blocks() -> 8 blokow

README.MD

```
# Todo API

A simple FastAPI-based Todo application that allows users to create, retrieve, update

## Features
- Create a new todo item
- Retrieve all todo items
- Update an existing todo item
- Delete a todo item

## API Endpoints
- POST /todos - Create a new todo item
- GET /todos - Retrieve all todo items
- PUT /todos/{id} - Update an existing todo item
- DELETE /todos/{id} - Delete a todo item

---

```yaml markpact:pipeline
description: Build a todo API
name: build-a-todo-api
steps:
- actor: script
 config:
 check: schema
 name: validate
- actor: script
 config:
 action: deploy
 target: docker
 name: deploy
version: 1.0.0
```

```text markpact:log
[2026-02-19T08:07:54Z] CONTRACT_CREATED: name=build-a-todo-api
```

```yaml markpact:orchestration
pipeline:
 name: todo-api
 steps:
 - name: validate_input
 actor: script
 - name: ai_review
 actor: llm
 - name: human_approval
 actor: human
 config: {channel: web, timeout: 7200}
```

WALIDACJA + OCZEKIWANIA

Znaleziono 8 blokow:

markpact:pipeline (214 chars)

markpact:log (62 chars)

markpact:orchestration (250 chars)

markpact:state (64 chars)

markpact:deps (35 chars)

markpact:file=app/main.py (1338 chars)

markpact:run (65 chars)

markpact:test (437 chars)

WALIDACJA

[V]

markpact:pipeline

lang=yaml, 214 chars

[V]

markpact:log

lang=text, 62 chars

[V]

markpact:orchestration

lang=yaml, 250 chars

[V]

markpact:state

lang=json, 64 chars

[V]

markpact:deps

lang=text, 35 chars

[V]

markpact:file=app/main.py

lang=python, 1338 chars

[V]

markpact:run

lang=bash, 65 chars

[V]

markpact:test

lang=text, 437 chars

### 3. Walidacja blokow

Sprawdzenie wymaganych blokow markpact:\*

README.MD

```
Todo API

A simple FastAPI-based Todo application that allows users to create, retrieve, update

Features
- Create a new todo item
- Retrieve all todo items
- Update an existing todo item
- Delete a todo item

API Endpoints
- POST /todos - Create a new todo item
- GET /todos - Retrieve all todo items
- PUT /todos/{id} - Update an existing todo item
- DELETE /todos/{id} - Delete a todo item

```yaml markpact:pipeline
description: Build a todo API
name: build-a-todo-api
steps:
- actor: script
  config:
    check: schema
    name: validate
- actor: script
  config:
    action: deploy
    target: docker
    name: deploy
version: 1.0.0
```

```text markpact:log
[2026-02-19T08:07:54Z] CONTRACT_CREATED: name=build-a-todo-api
```

```yaml markpact:orchestration
pipeline:
  name: todo-api
  steps:
    - name: validate_input
      actor: script
    - name: ai_review
      actor: llm
    - name: human_approval
      actor: human
      config: {channel: web, timeout: 7200}
```

WALIDACJA + OCZEKIWANIA

Wymagane bloki:

```
markpact:orchestration -- pipeline krokow
markpact:state -- stan procesu
markpact:deps -- zaleznosci
markpact:file -- kod zrodlowy
markpact:run -- komenda uruchomienia
```

Opcjonalne:

```
markpact:pipeline -- definicja steps
markpact:log -- historia zdarzen
markpact:test -- testy HTTP
markpact:target -- platforma docelowa
```

Wynik: PASS

WALIDACJA

[V]	markpact:orchestration obecny
	wymagany
[V]	markpact:state obecny
	wymagany
[V]	markpact:deps obecny
	wymagany
[V]	markpact:file obecny
	wymagany
[V]	markpact:run obecny
	wymagany
[V]	markpact:pipeline obecny
	opcjonalny
[V]	markpact:log obecny
	opcjonalny
[V]	markpact:test obecny
	opcjonalny
[?]	markpact:target
	opcjonalny, brak

4. Analiza zawartosci

Szczegolowa analiza kazdego bloku

README.MD

```
# Todo API

A simple FastAPI-based Todo application that allows users to create, retrieve, update

## Features
- Create a new todo item
- Retrieve all todo items
- Update an existing todo item
- Delete a todo item

## API Endpoints
- POST /todos - Create a new todo item
- GET /todos - Retrieve all todo items
- PUT /todos/{id} - Update an existing todo item
- DELETE /todos/{id} - Delete a todo item

---

```yaml markpact:pipeline
description: Build a todo API
name: build-a-todo-api
steps:
- actor: script
 config:
 check: schema
 name: validate
- actor: script
 config:
 action: deploy
 target: docker
 name: deploy
version: 1.0.0
```

```text markpact:log
[2026-02-19T08:07:54Z] CONTRACT_CREATED: name=build-a-todo-api
```

```yaml markpact:orchestration
pipeline:
 name: todo-api
 steps:
 - name: validate_input
 actor: script
 - name: ai_review
 actor: llm
 - name: human_approval
 actor: human
 config: {channel: web, timeout: 7200}
```

WALIDACJA + OCZEKIWANIA

Analiza blokow:

Deps: 4 pakietow

fastapi, uvicorn, sqlalchemy, pydantic

File: app/main.py (42 linii)

Run: uvicorn app.main:app --host 0.0.0.0 --port \${MARKP

Test: 6 assercji HTTP

WALIDACJA

[V] 4 zaleznosci

fastapi, uvicorn, sqlalchemy, pydantic

[V] file=app/main.py

42 lines, sha256=fdc3753d7475

[V] Run command

uvicorn app.main:app --host 0.0.0.0 --port \${MARKPACT\_PORT:-

[V] 6 testow HTTP

GET/POST/PUT/DELETE

# 5. Integralnosc SHA-256

Hash kazdego bloku markpact:\*

README.MD

```
Todo API

A simple FastAPI-based Todo application that allows users to create, retrieve, update

Features
- Create a new todo item
- Retrieve all todo items
- Update an existing todo item
- Delete a todo item

API Endpoints
- POST /todos - Create a new todo item
- GET /todos - Retrieve all todo items
- PUT /todos/{id} - Update an existing todo item
- DELETE /todos/{id} - Delete a todo item

```yaml markpact:pipeline
description: Build a todo API
name: build-a-todo-api
steps:
- actor: script
  config:
    check: schema
    name: validate
- actor: script
  config:
    action: deploy
    target: docker
    name: deploy
version: 1.0.0
```

```text markpact:log
[2026-02-19T08:07:54Z] CONTRACT_CREATED: name=build-a-todo-api
```

```yaml markpact:orchestration
pipeline:
  name: todo-api
  steps:
    - name: validate_input
      actor: script
    - name: ai_review
      actor: llm
    - name: human_approval
      actor: human
      config: {channel: web, timeout: 7200}
```

WALIDACJA + OCZEKIWANIA

Integralnosc danych:

README.md: 4c399f9f618fba6f7af5a832aeb00c0a

markpact:pipeline: 87bc50cbf04b17c8

markpact:log: 02a5b28c007207f3

markpact:orchestration: 53d7e4dd2e9e0244

markpact:state: 7e03664b9edc6836

markpact:deps: 1cf81989283d4bdc

markpact:file=app/main.py: fdc3753d7475af71

markpact:run: b49cce15b5712fdd

markpact:test: b6c417f1e1dc4bcc

Kazdy blok ma unikatowy hash.

Zmiana = nowy hash = wykryta.

WALIDACJA

[V] README.md hash

4c399f9f618fba6f7af5a832

[V] markpact:pipeline

sha256=87bc50cbf04b17c8

[V] markpact:log

sha256=02a5b28c007207f3

[V] markpact:orchestration

sha256=53d7e4dd2e9e0244

[V] markpact:state

sha256=7e03664b9edc6836

[V] markpact:deps

sha256=1cf81989283d4bdc

[V] markpact:file=app/main.py

sha256=fdc3753d7475af71

[V] markpact:run

sha256=b49cce15b5712fdd

[V] markpact:test

sha256=b6c417f1e1dc4bcc

Podsumowanie

8

blokow

markpact:*

13

passed

walidacja

0

failed

bledy

132.9s

czas

total

Kroki:

[V] LLM generate_contract() (132845ms)
ollama/qwen2.5-coder:14b, 132845ms, 2821 chars

[V] marksync enrich (2ms)
+2 blocks: pipeline, log

[V] parse_blocks() (0ms)
8 blocks

[V] validate markpact:orchestration
present

[V] validate markpact:state
present

[V] validate markpact:deps
present

[V] validate markpact:file
present

[V] validate markpact:run
present

[V] Analiza deps
4 packages: fastapi, uvicorn, sqlalchemy, pydantic

[V] Analiza file=app/main.py
42 lines, sha=fdc3753d7475

[V] Analiza run
uvicorn app.main:app --host 0.0.0.0 --port \${MARKPACT_PORT:-8000}

Kontrakt wygenerowany z 1 promptu -> 8 blokow markpact